

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS700 Research Report

Feasibility Analysis of Blockchain-Based Logging for NFV Forensic Integrity in 5G Environments

Student number: u23525119

Supervisor:
Mr. SM (Sheunesu) Makura

Samvit Prakash

October 2026

Abstract

The latest generation of telecommunication networks presented us with 5G networks, poising its ultra-high data transmission rates, reduced latency and network slicing. The introduction of these advantageous features came from the use of technologies such as edge computing and network function virtualization, but they also brought along new security and privacy challenges.

The introduction of these new technologies significantly complicates the 5G network landscape leaving room for security risks and vulnerabilities. In an effort to help mitigate these shortfalls of 5G networks we propose blockchain-based logging for NFV forensic integrity. We aim to guarantee the integrity and validity of logs in 5G networks to thereby assist in digital forensics and digital forensic readiness to attempt to mitigate the newfound security and privacy challenges introduced by a complex network landscape.

Keywords: Blockchain, Network Function Virtualization, 5G Networks, Blockchain-based Logging, Edge Computing, 5G Network Slicing, Digital Forensics, Digital Forensic Readiness

1 Introduction

Fifth Generation Wireless Access Networks, the latest generation of cellular networks, are the result of cutting-edge research and development in the attempt to offer ultra-high data transmission rates and massively reduced latency [1, 2]. Through studies of the existing literature, 5G was found to be the leading deployment mode, as well as exhibiting high improvement of its coverage in 2024, since then 5G networks still remain the latest generation of cellular networks, it is therefore reasonable to expect that the use of these networks and their applications has continued to improve in the last two years. 5G networks have enabled applications, which otherwise would not be feasible, from Internet of Things to Smart Cities and other domains [2]. These features that 5G networks bring are achieved through introducing heterogeneous ecosystems in which multiple stakeholder are provided the means to play roles in the telecommunications ecosystem as well as edge-computing and many more technologies [3, 2].

5G networks enable heterogeneous ecosystems through the use of many novel technologies including, edge-computing, Network Function Virtualization (NFV) and service based architecture [2]. Network function virtualization is the technology that enables the dynamic reallocation of network resources to allow for multiple operators to share the network despite not owning the physical infrastructure. This is known as 5G network slicing. Network slicing allows for operators to divide the physical network into customized virtual networks tailored for specific requirements and use cases. These virtual networks are in turn offered by the network to operators that do not own the physical infrastructure [3].

The use of edge computing (Also known as fog computing) within 5G networks serves to reduce latency and bandwidth, allowing for real-time data processing and decision making. Edge computing is a distributed system that stores and processes resources closer to the location of users or the source of the data, thereby preventing processing occurring at a centralized data-processing location. In the context of 5G networks, this infers that data processing takes place at the edge of the network, or in other words the locations closest to the user, this being the physical infrastructure used in 5G networks [4].

It is apparent that the feature rich offerings of 5G networks are brought about by the innovative use of multiple current and advanced technologies. However, this complicates 5G networks and their extensive areas of applications, and thereby introducing computer security and privacy challenges. Through the use of network slicing, operators can provide multiple custom virtual networks to operators, however this feature also allows for attackers to exploit these slices to carry out sophisticated attacks. Side channel attacks and cross-slice DoS attacks are among these. Through the leveraging of edge computing, 5G networks reduce latency and bandwidth, however by moving processing (and thereby storage too) to the network's edge devices (the physical infrastructure) they introduce increased attack surfaces. With the increasing complexity of 5G networks and attack vectors, existing computer security technologies are falling short of the security requirements that 5G networks need [1].

Blockchain was originally developed for the inception of digital currencies, since then it has expanded its original domain of application by demonstrating significant potential in computer security and privacy applications [5, 4, 3]. Blockchain is akin to an immutable digital ledger where each entry in this ledger, once made, is permanent and unchangeable [5, 4]. Unlike tra-

ditional ledgers, blockchain technology is distributed across numerous nodes within a network. What makes blockchain have potential in computer security and privacy domains is the fact that all transactions are validated through consensus protocols, wherein each node validates the transaction. Altering any entry would require changing the same entry on each node on the network. This is an impractical task under realistic attack scenarios [5].

5G networks introduce new and innovative features, but at the same time introduces new computer security and privacy challenges [1]. Digital forensic readiness is not only a security need but also a core part of business logic. With these increased challenges and concerns, the need for digital forensics, digital forensic readiness and digital evidence becomes increasingly crucial.

Much research has already been done into 5G networks and its security landscape, however little research has gone into solutions for guaranteeing the validity and integrity of logs in the NFV architecture within 5G environments for forensic purposes. This paper therefore attempts to close the research gap by proposing a solution to maintain NFV log's validity and integrity in 5G environments for forensic purposes with the use of blockchain. We attempt this by:

- Performing a literature review of the existing 5G/NFV and Blockchain research landscape.
- Designing and implementing a prototype and research environment.
- Performing systematic experiments on our aforementioned prototype.

This research paper is proposed with the aim of proposing a new form of security within 5G environments, but also with the hope that this paper will create a new research niche into the field of 5G network security research.

This paper is structured as follows: Section 2 presents a literature review of 5G networks, digital forensics and digital forensic readiness, the importance of logs, blockchain and Hyperledger Fabric. Section 3 describes the design and implementation of the prototype and the simulated NFV environment in which it is deployed. Section 4 describes the experiments conducted on the prototype and presents their results. Section 5 provides a critical evaluation of these results through regression modeling, a visual analysis of resource utilization and an assessment of feasibility. Finally, Section 6 concludes the paper and discusses future work.

2 Literature Review

We first begin by performing a literature review into the current research landscape of NFV/5G environments. This literature review was performed with the intent of understanding the current architecture, needs, as well as background knowledge in order to implement our prototype and experimentation.

2.1 5G Networks

5G networks were designed and implemented with the goal of overcoming the limitations of the previous mobile network generations [6]. We therefore start our review on 5G networks by briefly analyzing the predecessors of 5G. The first generation (1G) was launched in the 1980's, and it was characterized by low-quality voice communications. Analog voice was the main technology backing 1G. 1G's ability was simple voice calls. The second generation (2G) was launched in the 1990's, and it was characterized by digital voice, SMS and improved security. The main technology backing 2G was GSM and allowed for text messages and calls. The third generation (3G) was launched in the 2000's and was backed by UMTS/WCDMA. It was best characterized by the introduction of mobile data and the mobile internet. It had the ability for web browsing and basic multimedia. The fourth generation (4G) was launched in the 2010's, and was characterized its high-speed data, and HD video streaming. 4G had the ability for streaming, video conferencing and apps, enabled through LTE [6]. From this brief glimpse into mobile network generations we can see a clear pattern of iterative improvement from one generation to the next, improving and innovating on the shortfalls of their predecessors.

As stated above, 5G networks were designed to overcome the limitations of previous mobile generations, offering high speeds (up to 10 Gbps), low-latency (under 1 ms), greater capacities (up to 1 000 000 connected devices/km^2) and flexible support for diverse use cases [6, 7]. Applications in current times are now producing massive amounts of data and require stronger requirements for reliability as well as security which previous mobile network generations could not provide [8]. The fifth generation of mobile networks, the next iterative generation following fourth generation mobile networks, provides dynamic, and programmable network architecture through network softwarization enabling a plethora of applications [6, 7]. These diverse applications would never have been feasible with the previously fixed and inflexible conventional networks [7]. 5G networks have brought us into a new era of

ultra-high data speeds, ultra-low latency and massive connectivity, allowing for a plethora of new and diverse, never seen before, applications [7]. To name a few applications that 5G can support are, enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and Massive Machine-Type Communications (mMTC) [6].

2.1.1 Network Slicing

Network slicing is a term that is consistently encountered when reading about 5G networks. It is one of the core innovations and supporting pillars of 5G networks. Network slicing allows for numerous independent virtual networks to operate over a single physical infrastructure, allowing for high efficiency and flexibility. It is a logical segmentation technique or method that is used to create independent virtual networks (slices) over a single physical infrastructure. Network slicing is highly flexible and configurable, resulting in operators being offered highly tailored slices meeting their unique business requirements. These slices can be dynamically created, adjusted and deactivated in real time allowing for dynamic scalability. Each network slice is given dedicated resources in order to maintain consistent performance across all virtual networks [6]. Network slicing allows for unprecedented benefits, however introduces new security risks and complexities simultaneously.

As established above, network slicing is an innovation providing 5G networks with unprecedented benefits. Network virtualization banks on three major technologies, SDN, NFV, and cloud computing [7]. For this literature review we will only be discussing the first two technologies as they are closely related to the work that we will be proposing.

2.1.2 Software Defined Networking (SDN)

Software defined networking (SDN) is a network architecture that allows us to decouple network management from the network traffic flow in order to provide optimized network services. The architecture of SDN is comprised of three layers:

- Infrastructure Layer or Data Plane
- Control Layer/Plane
- Application Layer/Plane

The infrastructure layer contains the physical network elements that handle the computing and forwarding capabilities of the network. The control

layer contains the SDN controller which manages the network's traffic flow through its many routers and switches. The Control Layer is also responsible for processing the instructions passed down from the application layer and propagated instructions down to the Infrastructure Layer. The application layer contain the programs that communicate with the Control Layer. This architecture allows for the dynamic scaling-up and down of networks. SDN allows network operators to install specific security policies for each network [8].

The SDN architecture has four main properties:

- Programmable Networks
- Agile Networking
- Logically Centralized Networking
- Open Networking

Programmable networks are possible due to the decoupling of the control plane from the data. Due to this decoupling, the network behavior is thereby controlled through the application software, which in turn allows us to automate configurations of network functions and can tailor the network for specific services. Decoupling also allows for agile networking, by enabling network operators to dynamically modify network traffic to meet business requirements. SDN's architecture is based on logically centralized network topologies, which allow for for logically centralized networking with intelligent control and orchestration of network resources. SDN uses open standards to provide open networking, which in turn allows for flexible and adaptable network service. SDN and NFV are complementary technologies [8].

2.1.3 Network Function Virtualization (NFV)

The European Telecommunications Standards Institute (ETSI) defined the NFV architectural framework, with three distinct layers: the NFV Infrastructure (NFVI) layer, the Virtual Network Functions (VNFs) layer and the NFV Management and Orchestration (NFV-MANO) layer [9, 8, 10].

Alnaim[8] discusses each of the NFVI layers. It discusses three units that form the NFVI layer, the hardware elements, the virtual elements and the virtualization layer. The hardware elements of the NFVI is made up of compute, storage and network resources. The virtual elements of the NFVI is made up of virtual compute, virtual storage and virtual network resources.

The virtualization layer includes the Virtual Machine Manager (VMM), also known as the hypervisor. The hypervisor's responsibilities include emulating hardware resources and ensuring the clear isolation between virtual machines [9, 8].

The VNFs are essentially just software packages that implement network functions, and are emulated through the NFVI. The VNFs are deployed across multiple virtual machines and can contain one or more Virtual Network Components (VNFC) [8].

The NFV-MANO automatically provides management and orchestration to NFVI and NFV resources, as well as life-cycle management and the management of NFV and Network Services (NSs). The responsibilities of the NFV-MANO are handled by three MANO units, the Virtual Infrastructure Manager (VIM), The VNF Manager (VNFM), and finally the NFV Orchestrator (NFVO). The VIM unit is responsible for managing the NFVI resources and controlling the interactions between the NFVI and NFV layers. Additionally it collects and forwards performance measurements from all the MANO units and forwards them to other MANO units. The VNFM has responsibilities pertaining to the management of the VNFs life-cycles. This can include the creation and removal of VNF images, scaling VNF resources, updating VNF statuses, and more. The NFVO performs orchestration and authorization for resources needed by network functions. Their responsibility includes managing their life-cycle, which also includes scaling, terminating and collecting performance measurements from VIM and VNFM units [8, 9].

2.1.4 Security Challenges in 5G/NFV Environments

5G's transformation on global connectivity introduced higher speeds, greater capacity and support for various applications scenarios, however it necessitates an extremely complex security environment in comparison to its predecessors [6, 10]. NFV enables flexible networks and rapid service creation, yet with each benefit and security opportunity brought forth, comes a new security challenge and increase in complexity. The European Union Agency for Cybersecurity (ENISA) identified and described 60 security challenges, and further categorized these challenges across seven categories in a bid to make them more understandable [10].

Network slicing is one of 5G's core features, allowing for independent virtual networks to operate on shared infrastructure. This innovation brought great efficiency and flexibility, however introduces new security concerns. The

implementation of network slicing uses technologies such as Software Defined Networking (SDN) and NFV, however these technologies expand the attack surface and risk vulnerabilities in confidentiality, integrity and availability. Common attack categories include DoS, unauthorized access, data interception, API exploitation and configuration tampering. Many of these attack categories are amplified in impact and effectiveness due to virtualised and multi-tenant environments, the core features of 5G [6, 10].

Organizational challenges and Legacy Technologies play a role in the secure deployment and operations of network slicing. These include compliance with standards, integration with legacy infrastructure and regulatory obligations [6].

Intensive investigations and research into these challenges is yet to be done and hinders the development of effective solutions [10, 6]. The challenges brought with 5G are issues that have not been seen in its predecessors, the current standards and security mechanisms in place are ill equipped to handle the new security challenges brought forth with 5G.

2.2 Digital Forensics and Digital Forensic Readiness

Digital forensics is the discipline concerned with the identification, collection, preservation, analysis and presentation of digital evidence so that it may be used to investigate incidents and support legal proceedings. Casino et al. [11] performed a review of reviews of the digital forensics literature, and found that the field continues to be challenged by the volume, variety and volatility of digital evidence, as well as the lack of standardization across tools and processes. These challenges are amplified in modern virtualised and distributed environments, where evidence is spread across heterogeneous components that investigators cannot always physically access [12].

To bring consistency to the investigation process, the international standard ISO/IEC 27043 [13] defines incident investigation principles and processes. The standard describes a set of process classes, which begin with readiness processes that are performed before an incident occurs, followed by initialization, acquisitive and investigative processes that are performed once an incident is detected. The inclusion of a readiness class highlights that the quality of an investigation is largely determined by the preparations made before an incident takes place.

This preparation is referred to as digital forensic readiness (DFR), which is the ability of an organization to maximize its capacity to collect credible digital evidence while minimizing the cost of an investigation. Grispos et al. [14] argue that systems should be engineered to be forensic-ready from their inception, rather than having forensic requirements retrofitted after an incident. In this view, forensic readiness is a property of the system, and one that must be considered during design alongside other non-functional requirements.

Machaka and Balan [12] investigated proactive digital forensics in cloud and networked environments. Their findings show that traditional, reactive investigations are becoming obsolete due to rapid technological change, and that proactively collecting evidence improves both the speed and accuracy of forensic activities. They also note that the legality of live and remote forensics remains a concern in several jurisdictions, which can affect the credibility of the evidence obtained.

The need for readiness extends to software-defined and virtualised network infrastructure. Karie and Valli [15] examined the issues and challenges of implementing digital forensic readiness in Software Defined Networks (SDN), a technology on which 5G networks rely. Their work emphasizes that the programmable and centralized nature of these networks introduces forensic challenges that traditional network forensics did not need to address.

Forensics in cellular networks specifically has received less attention. Sharevski [16] reviewed the forensic mechanisms available in LTE and LTE-Advanced networks, such as lawful interception and user localization, along with their associated evidence types, tools and legal frameworks. The author then discusses how 5G technologies, including NFV, network slicing and CUPS, require adaptations before these capabilities can be retained in 5G networks. Building on this, Makura and Venter [17] work towards a digital forensic readiness model for 5G NFV environments, and identify DFR as a necessary step towards enabling investigations in these environments.

Together, these works show that forensic readiness is an established requirement, and that 5G NFV environments pose new difficulties for meeting it. A recurring prerequisite across them is the availability of trustworthy evidence, of which logs are the primary source. The next section therefore discusses the role of logs and the need to protect their integrity.

2.3 Importance of Logs

Logs are the primary record of events within a computing system, and are therefore among the most important sources of digital evidence. In NFV/5G environments, logs are generated across the NFVI, VNF, NFV-MANO, network slice, control plane and security layers [17]. Their forensic value, however, depends on whether they can be trusted. If an attacker is able to modify, delete or fabricate log entries, the logs lose their value as evidence and may mislead an investigation.

This problem has motivated research into tamper-evident logging. Zhao et al. [18] revisit tamper-evident logging and propose a high-performance auditing system that is co-designed to address the cost of securing logs at scale. Their work highlights that the overhead of conventional tamper-evident schemes has limited their practical adoption, and that logging systems must provide integrity guarantees without degrading system performance. This is a relevant concern for 5G environments, which generate high volumes of log data.

Blockchain technology has been proposed as an alternative means of securing logs. Pourmajidi et al. [19] present immutable log storage as a service on both private and public blockchains. Their work demonstrates that the immutability properties of a blockchain can be used to protect logs from alteration, and it compares the trade-offs of deploying such a service on private as opposed to public chains.

Islam and Rahman [20] propose LogStamping, a blockchain-based log auditing approach for large-scale systems. Their approach targets the scalability challenges of recording logs from large systems, and shows that blockchain-based auditing can be applied to systems that generate large amounts of log data.

These works show that the integrity of logs can be strengthened through tamper-evident and blockchain-based mechanisms, while also showing that performance and scalability remain significant concerns. Neither, however, has been examined in the context of the NFV architecture of 5G networks, which is the gap this research addresses.

2.4 Blockchain

A blockchain is a distributed ledger in which records are grouped into blocks that are cryptographically linked to the preceding block, forming an append-

only chain [5, 4]. The ledger is replicated across the nodes of a network, and new blocks are added only after the nodes agree on their validity through a consensus protocol. As a result, altering a recorded entry would require altering all subsequent blocks and gaining the agreement of the network, which makes tampering evident and, under realistic attack scenarios, impractical [5].

Alghuried et al. [5] survey the security and privacy threats, challenges, applications and tools of blockchain systems. They show that, while blockchain offers strong integrity guarantees, it is not immune to attack, and that its security depends on factors such as the consensus mechanism, smart contract correctness and the underlying network. Similarly, Nezami et al. [4] reviewed the literature on the nexus of blockchain and edge computing. Their review is relevant to 5G networks, in which edge computing is a core feature, as it examines how blockchain can provide trust and security in distributed edge environments.

Blockchains are commonly classified as public or permissioned. Public blockchains allow any participant to join and validate transactions, while permissioned blockchains restrict participation to identified and authorized entities. Afraz et al. [3] analyse the requirements and costs of applying blockchain and smart contracts to telecommunications. They find that permissioned blockchains are well suited to telecommunications ecosystems, in which multiple known stakeholders must cooperate, but that scalability and the cost of adoption remain significant considerations.

Blockchain has also been applied directly to logging and forensics. Pourmajidi et al. [19] use blockchains to provide immutable log storage, and Islam and Rahman [20] introduce LogStamping for auditing the logs of large-scale systems. Akbarfam et al. [21] propose ForensiBlock, a provenance-driven blockchain framework for data forensics and auditability, which uses the blockchain to record the provenance of evidence so that its handling can be verified. These works show that blockchain can provide the integrity and auditability that forensic processes require.

Based on the above, a permissioned blockchain is the most suitable model for securing the logs of NFV environments, as the participants are known entities and the logs must be validated and stored with limited overhead. The following section introduces Hyperledger Fabric, the permissioned blockchain platform used in our prototype.

2.5 Hyperledger Fabric

Hyperledger Fabric is an open-source, permissioned blockchain platform hosted by the Linux Foundation [22]. Unlike public blockchains, Fabric requires all participants to have a verifiable identity, and it does not rely on a cryptocurrency. Smart contracts, known as chaincode, can be written in general-purpose languages such as Go, Java or JavaScript [22]. Fabric uses a modular architecture in which components such as the consensus mechanism and the membership services can be replaced to suit different use cases [22].

Fabric also introduced the execute-order-validate transaction architecture, in contrast to the order-execute architecture of many earlier blockchains [22]. In the execution phase, transactions are simulated by endorsing peers. In the ordering phase, the transactions are placed into a total order and packaged into blocks. In the validation phase, peers validate the transactions and commit the blocks to their ledgers. This separation allows transactions to be executed in parallel and improves throughput [22, 23]. The components that realize this architecture are discussed in the following subsections.

2.5.1 Peers

Peers are the nodes that host the ledger and chaincode, and they are the foundation of a Fabric network [22]. Each peer maintains a copy of the ledger, which consists of the blockchain and a world state database holding the current values of ledger entries. Peers may fulfill different roles. Endorsing peers execute chaincode to simulate transactions and sign the results, while committing peers validate and commit the ordered blocks to the ledger [22]. Peers are owned by the organizations in the network, and blocks are distributed among the peers of an organization through a gossip protocol. In the context of this research, peers would be operated by the entities that need to record and verify NFV logs.

2.5.2 Orderers

Orderers form the ordering service, which establishes the order of transactions and packages them into blocks that are distributed to the peers [22]. Orderers do not execute chaincode or maintain the world state, and so consensus is separated from execution. Fabric has supported several ordering implementations, including Kafka-based ordering [24] and Raft, a crash fault tolerant consensus protocol, while Sousa et al. [25] demonstrated a Byzantine fault tolerant ordering service. The ordering service is critical to log

integrity, as it determines the order in which log records are committed.

2.5.3 Certificate Authority and Membership Services

Fabric's permissioned nature is enforced through its identity management. Each participant is issued an X.509 digital certificate by a Certificate Authority (CA), and the Membership Service Provider (MSP) uses these certificates to determine which identities are valid and what roles and permissions they hold [22]. Fabric provides its own CA, Fabric CA, for registering and enrolling identities. This ensures that every log submission can be attributed to a known entity, which supports the accountability that forensic processes require.

2.5.4 Channels

Channels are private communication sub-nets within a Fabric network, in which only the member organizations of a channel can take part [22]. Each channel has its own separate ledger, and so data on one channel is not visible to organizations that are not members. Channels therefore enable confidentiality between parties that share a network. This is relevant to multi-tenant 5G environments, as logs belonging to different operators or slices could be separated onto distinct channels.

2.5.5 Chaincode Builders and Docker

Chaincode must be built and run before it can be invoked on a peer. In Fabric's default configuration, the peer uses Docker to build the chaincode into an image and to launch it in an isolated container, separate from the peer process [22, 23]. This isolation prevents the chaincode from interfering with the peer, and it simplifies the deployment of the network. Fabric also supports external builders as an alternative to the Docker-based approach. Our prototype uses the default Docker-based builder, which allows the Fabric network and chaincode to be deployed in a reproducible containerized environment.

3 Prototype Design and Implementation

This section describes the artifacts developed for this research. These are a blockchain logging Virtual Network Function (VNF), a simulated 5G NFV environment in which the VNF is deployed, and an experiment that evaluates

the VNF within that environment. Section 3.1 presents the design of the VNF, and Section 3.2 presents the simulated environment. The complete implementation is available in a public repository.[1]

3.1 VNF Design

The prototype is a VNF that records the logs of 5G network components on a Hyperledger Fabric blockchain, so that the integrity and validity of these logs can later be verified for forensic purposes. Hyperledger Fabric was selected because it is a permissioned blockchain [22]. Every participant requires an identity issued by a Certificate Authority (CA), which means that every log can be attributed to a known entity. Each peer is enrolled with its own CA, MSP and TLS credentials, and takes part only in the channels (shared ledgers) to which it has been assigned. The prototype therefore provides credential and identity provisioning for all participants, as well as centralized management of channel membership.

3.1.1 Overview and Placement in the NFV Architecture

The prototype VNF consists of two parts, illustrated in Figure 1:

- **BCL** (Blockchain Logging), the logic deployed at every physical site; and
- **BCLm** (Blockchain Logging management), the management logic deployed once at the central data center.

Each component is realized as one or more Docker containers. The prototype was designed to fit into the ETSI NFV reference architecture [26, 8, 9], as shown in Figure 2. BCL is a VNF. BCLm is not a VNF Manager (VNFM), as a VNFM manages the life-cycle of VNF instances, whereas BCLm manages the application-level function of BCL by configuring the Fabric network on which all BCL instances depend. BCLm is therefore placed as the centralized Element Manager (EM) of BCL [26]. Within BCLm, the component named VNFM is a prototype-specific manager of the Fabric deployment, and should not be confused with the VNFM of the NFV-MANO.

3.1.2 BCL: Site-Level Components

BCL is deployed once per physical site, alongside the gNB that it serves, and consists of four containers.

[1] `https://github.com/SamvitPrakash/Blockchain-Logging-Prototype`

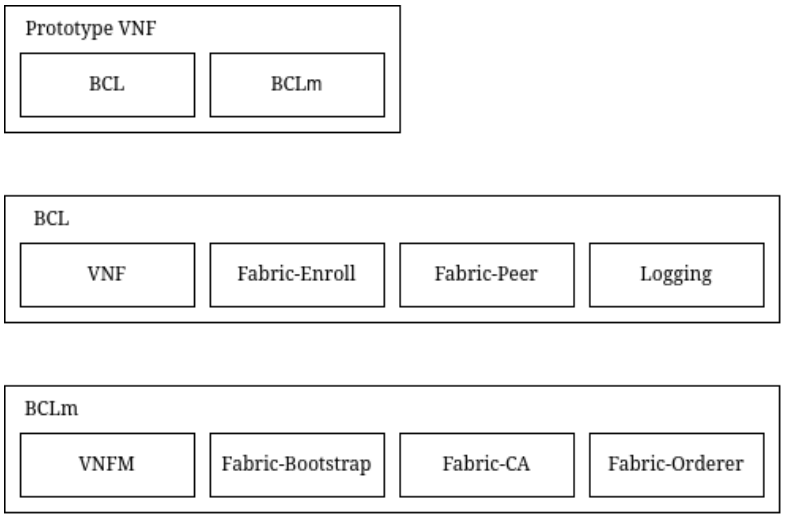


Figure 1: Components of the prototype VNF. BCL runs at each physical site and BCLm runs at the central data center.

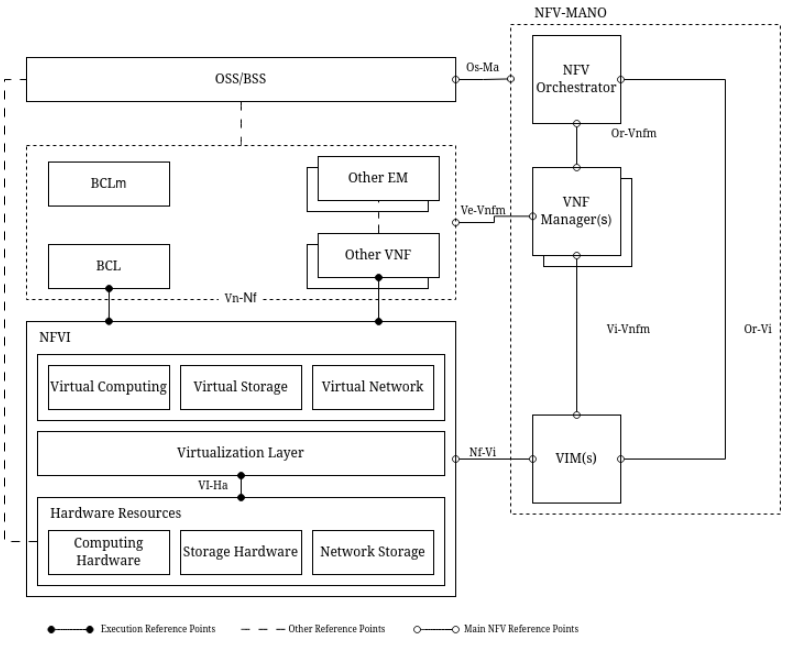


Figure 2: Placement of the prototype within the ETSI NFV reference architecture. BCL is a VNF and BCLm is its centralized EM.

VNF captures the logs of the gNB, prepares them for the blockchain and sends them to the local Fabric-Peer to be committed. It obtains its own credentials and identity from the Fabric-CA, which allows it to sign the logs that it submits and thereby attribute every log to a known entity. The VNF communicates with the gNB over an OAM network that is created for each gNB–VNF pair.

Fabric-Enroll provisions the credentials and identity of the corresponding Fabric-Peer from the Fabric-CA, as a Fabric peer cannot start without them. Once provisioning is complete, the container terminates and Docker Compose starts the Fabric-Peer container. In a real deployment, this start-up orchestration would be the responsibility of the NFV-MANO.

Fabric-Peer is the unmodified, open-source Hyperledger Fabric peer [22]. On start-up, it coordinates with the VNFM in BCLm to determine the chaincode builder and logging package, after which it starts the logging container. The peer provides the gateway through which the VNF sends its prepared logs, which it commits to the ledger of the channels to which it belongs. It receives new blocks from the Fabric-Orderer, either directly or from other peers through gossip [22].

Logging is started by the Fabric-Peer once the chaincode builder and logging package have been agreed upon with the VNFM.

3.1.3 BCLm: Central Management Components

BCLm is deployed once, at the central data center, and manages the Fabric network that is used by all BCL instances.

VNFM sets up the Fabric deployment. It assigns peers to channels using a pseudo-random assignment that is driven by a seed value, which makes a given blockchain topology reproducible. It then coordinates the chaincode builder and logging package with the peers of the BCL instances. Only the peers assigned to a channel can take part in the ledger of that channel [22]. The VNFM also obtains its own credentials and identity from the Fabric-CA.

Fabric-Bootstrap fulfills the same role for the orderer that Fabric-Enroll fulfills for the peers. It provisions the credentials and identity of the Fabric-Orderer from the Fabric-CA and then terminates, after which Docker Compose starts the Fabric-Orderer.

Fabric-CA is the open-source Hyperledger Fabric Certificate Authority. It issues and manages all credentials and identities in the prototype, including those of the peers, the orderer, the VNFs and the VNFM [22].

Fabric-Orderer is the open-source Hyperledger Fabric ordering service [22]. It orders the transactions of each channel, packages them into blocks and delivers these blocks to the peers of that channel. The ledgers themselves are held by the peers.

3.1.4 Configurability and Scalability

The prototype was designed so that deployments are changed through parameters rather than through code. The number of physical sites can be scaled to any number without code changes, limited only by the resources of the host and the address space available for network and sub-net creation. Likewise, any blockchain topology can be created by changing parameters, so that any number of peers can be deployed and can take part in any number of channels. The build scripts assign peers to channels pseudo-randomly, which ensures that only the assigned peers take part in each channel.

3.1.5 Separation from Normal Network Operations

The prototype is separated from normal 5G operations at the network level. BCL instances communicate with each other and with BCLm over a dedicated network (the XIT network), rather than over the networks used by the 5G core. The only point of contact with normal operations is the gNB–VNF OAM link over which the logs are obtained. Should a gNB be compromised, the only prototype container reachable from it is the VNF. Should the VNF in turn be compromised, the XIT network would be exposed to some risk as well.

3.2 Simulated NFV Environment

3.2.1 Real-World Deployment

Figure 3 shows how the prototype would be deployed in a real 5G NFV network. BCLm resides in the central data center as an EM, next to the 5G core network functions, while the NFV-MANO and OSS/BSS manage the infrastructure. Each physical site hosts a gNB and a BCL instance on its own NFVI, and the number of sites can grow arbitrarily.

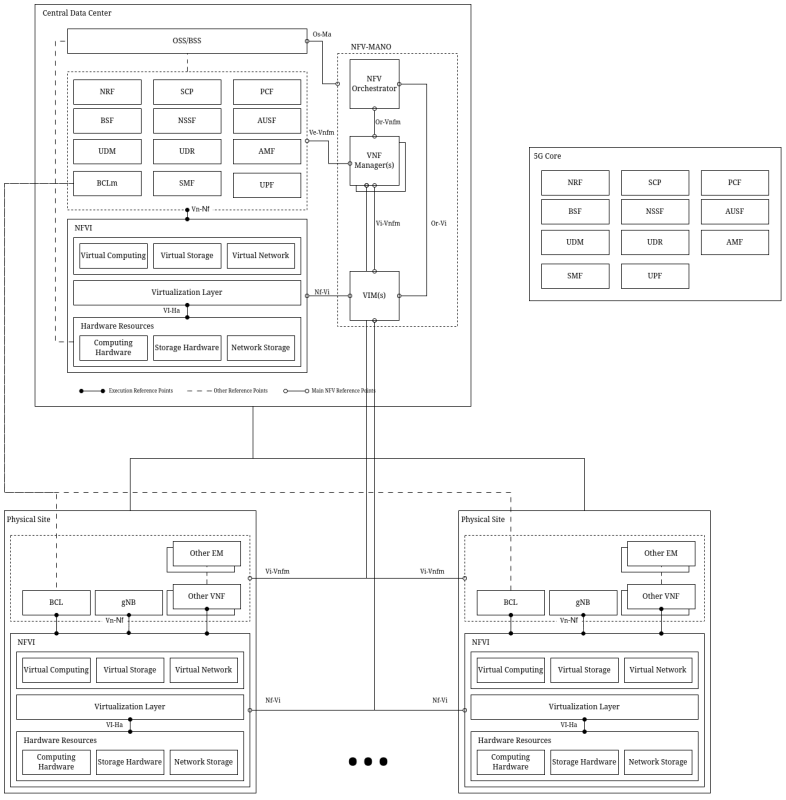


Figure 3: Basic real-world deployment of the prototype.

3.2.2 Simulated Deployment

The aim of this research is to evaluate the prototype and not to build a production 5G network, and the NFV environment was therefore simulated. The 5G core is provided by Open5GS [27], an open-source implementation of the 5G core network functions, deployed through the `docker_open5gs` project [28], which packages each network function as a Docker container. The gNB and user equipment (UE) are provided by a RAN simulator that is bundled with `docker_open5gs`. Containers provide lightweight, operating-system-level virtualization and are therefore a practical substitute for the virtual machines of the NFVI in a simulated environment. The prototype was built in the same way, with each component packaged as a container image, and Hyperledger Fabric is likewise distributed as Docker images.

Figure 4 shows the simulated deployment. The central data center hosts BCLm and the Open5GS 5G core (NRF, SCP, PCF, BSF, NSSF, AUSF, UDM, UDR, AMF, SMF and UPF), while each physical site hosts a simulated gNB and a BCL instance. The NFVI and NFV-MANO are abstracted by Docker and Docker Compose, supplemented by bash scripts that perform the orchestration and life-cycle management that the NFV-MANO would normally perform. The OSS/BSS is not implemented.

3.2.3 Networking

The simulated environment uses three types of network, summarized in Table 1.

Network	**Purpose**
Docker bridge network	Communication between the Open5GS containers.
OAM	Communication between each gNB–VNF pair, with one OAM network per pair.
XIT	Communication between the instances of the prototype (BCL and BCLm).

Table 1: Networks used in the simulated environment.

The OAM network is an existing NFV network and is used here for its usual purpose of operations, administration and maintenance. The XIT network is modeled on the Xn interface, which connects gNBs to each other in

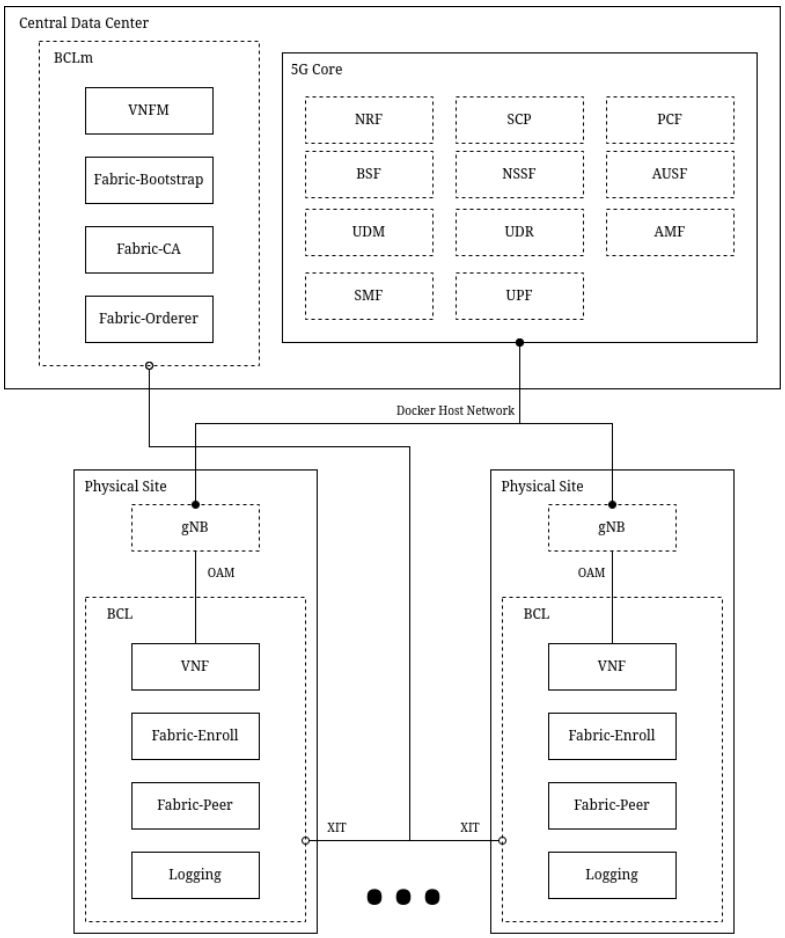


Figure 4: Simulated deployment used in this research. Docker and scripts abstract the NFVI and NFV-MANO.

the 5G RAN [29]. The Xn interface cannot be reused as is, as the prototype requires communication between prototype instances rather than between gNBs, and the XIT network was therefore created.

3.2.4 Limitations of the Prototype

As the prototype exists to evaluate the concept, several deliberate compromises were made.

Abstracted management layers. The NFVI and NFV-MANO are abstracted by Docker and scripts, and some of their duties are therefore performed manually. Some containers are given access to the Docker daemon so that they can start other containers dynamically, which the NFV-MANO would normally do. This access is equivalent to root access on the host, and would not be acceptable in a production deployment.

Log capture. The VNF reads the log file inside the gNB directly. A real deployment would require a log provisioning mechanism to deliver logs to the VNF.

Archival and rotation. The ledger of a Fabric channel is append-only and grows continuously, and a blockchain is not intended to serve as bulk log storage. A real deployment would therefore require log archival and rotation, in which BCLm retrieves logs into a central database while the blockchain retains the information needed to verify them later. This mechanism is not implemented.

4 Experimentation

The experimental evaluation was conducted to investigate the computational resource requirements of the proposed prototype under varying network configurations. Specifically, the experiments aimed to determine how the number of deployed 5G towers and the number of shared blockchain ledgers influence the CPU utilization, memory consumption, network traffic, and disk I/O of the prototype's constituent components.

Each experiment consisted of the deployment and execution of a complete simulated network topology, followed by its teardown. A topology was defined by two independent configuration parameters: the number of towers

and the number of shared ledgers. During execution, an independent monitoring process continuously collected resource utilization measurements from the Docker containers associated with the prototype.

The experiments were designed to evaluate the prototype under different deployment scales while maintaining a consistent execution procedure. Each topology was deployed with the objective of establishing the required network and blockchain functionality, including the successful commitment of a known transaction to the corresponding ledger for every deployed tower.

The collected measurements were subsequently used to characterize the computational overhead associated with individual components and to investigate how resource requirements change as the number of towers and shared ledgers increases.

4.1 Experiment Design

The experiments followed a systematic parameterized design in which the number of towers and shared ledgers were varied to produce different network topologies. Each topology represented a distinct experimental configuration, allowing the resource requirements of the prototype to be evaluated under progressively larger deployments.

4.1.1 Experimental Parameters

Two configuration parameters were varied throughout the experiments:

- **Number of towers (T):** The number of simulated 5G tower sites deployed within the network. Each tower was associated with its respective 5G network components and blockchain client functionality.
- **Number of shared ledgers (L):** The number of distinct shared blockchain ledgers configured within the network. Each shared ledger was associated with a group of participating towers.

The number of towers was varied from a minimum of two to a maximum of n, where the upper limit was determined by the computational resources available on the experimental host machine ($n = 31$ in the conducted experiments).

For each tower count, the number of shared ledgers was varied subject to

the constraint that every ledger required at least two participating towers. Consequently, only configurations satisfying the following condition were considered:

$$1 \leq L \leq \left\lfloor \frac{T}{2} \right\rfloor \tag{1}$$

where T denotes the number of towers and L denotes the number of shared ledgers.

Configurations violating this constraint were excluded from the experimental sequence. This ensured that each generated topology satisfied the minimum participation requirement for its shared ledgers.

The valid configuration space can therefore be expressed as:

$$\mathcal{C} = \left\{ (T, L) \;\middle|\; 2 \leq T \leq n,\ 1 \leq L \leq \left\lfloor \frac{T}{2} \right\rfloor \right\} \tag{2}$$

Each valid configuration was assigned a unique experiment identifier, allowing the collected measurements to be associated with the corresponding network topology.

4.1.2 Experimental Procedure

The experiments were conducted using an automated execution procedure designed to evaluate the prototype across different network configurations. Each experiment represented a complete deployment life cycle of a single network topology, rather than a modification or extension of an existing deployment.

This distinction was important because it ensured that the resource measurements captured the computational overhead associated with establishing and operating the entire prototype, including the initialization of the network components and blockchain infrastructure.

The experiments were executed sequentially, with each valid combination of towers and shared ledgers representing a distinct experimental configuration. The procedure consisted of the following stages:

1. **Configuration:** The number of towers and shared ledgers was established for the current experiment. A unique experiment identifier and the corresponding topology parameters were written to a shared

state file, allowing the resource monitoring process to associate collected measurements with the active configuration.

2. **Prototype teardown:** Before generating the new topology, any existing prototype deployment was torn down. This removed the previously deployed prototype environment, ensuring that the subsequent configuration could be established independently rather than reusing components from an earlier experiment.

3. **Topology generation and deployment:** The topology generation script was executed using the specified tower count, shared-ledger count, and generation seed. The required 5G network components and blockchain infrastructure were instantiated according to the generated configuration.

4. **Network establishment:** The deployment automation monitored the final tower's associated VNF for confirmation that the UE's PDU session resources had been successfully established. This indicated that the network had reached the expected operational state.

5. **Transaction verification:** The functional completion objective was the successful commitment of a known transaction to the corresponding shared ledger for every deployed tower. This provided a consistent functional criterion against which the execution of each topology could be evaluated.

6. **Settling period:** Following successful network establishment, an additional settling interval of 60 seconds was provided. Resource measurements continued throughout this interval to capture the behavior of the deployed components following initial startup activity.

7. **Experiment completion:** Following completion of the execution procedure, the experiment was considered concluded. The next experimental configuration could then begin, repeating the teardown, generation, deployment, and execution process from a fresh prototype state.

A separate resource monitoring process operated continuously throughout the experimental sequence. Unlike the prototype deployment, this process was not restarted between experiments. Consequently, resource measurements were collected across consecutive deployment life cycles, including the periods in which containers were created, operated, and removed.

The experimental automation systematically varied the topology parameters, beginning with the smallest supported configuration and progressing towards larger deployments. For each tower count, the valid shared-ledger configurations were evaluated sequentially.

The topology generated for each experiment was also preserved as a JSON file, providing a record of the network configuration associated with the collected resource measurements.

After all configurations had been evaluated, the monitoring process was terminated and a final cleanup procedure was executed to remove the remaining experimental environment.

4.1.3 Experiment Completion Condition

A distinction was made between the successful deployment of a topology and the successful completion of its intended operation.

The establishment of the simulated 5G network, including the successful creation of the required PDU session resources, indicated that the network components had reached the expected operational state. However, successful network establishment alone did not necessarily demonstrate that the blockchain functionality had completed its intended task.

The intended functional completion condition was therefore defined as the successful commitment of a known transaction to its corresponding shared ledger for every deployed tower.

This criterion ensured that each experimental topology was evaluated against a consistent functional objective, rather than terminating solely after a predetermined execution duration.

In addition, the deployment automation employed a fixed settling interval following the detection of network establishment. This allowed resource measurements to continue after the initial deployment activity.

4.1.4 Resource Monitoring and Data Collection

Resource utilization was measured using an independent monitoring script that operated concurrently with the deployment automation. The monitoring process was initiated before the automated sequence of topology configura-

tions and remained active throughout the experiments.

The monitoring process queried the Docker runtime using the `docker stats` command at a nominal sampling interval of one second. This approach enabled resource measurements to be collected directly from the containers implementing the prototype without requiring modifications to the prototype's application logic.

During each sampling iteration, the logger identified containers belonging to the prototype using their Docker Compose project information. Additional discovery logic was implemented to include dynamically instantiated Hyperledger Fabric chaincode containers, launched by the peers through the Docker-based builder, that did not expose the same Docker Compose labels.

The monitoring process periodically reloaded the shared experiment state file, which contained the active experiment identifier, tower count, and shared-ledger count. Consequently, measurements collected during different topology configurations could be distinguished without restarting the logger.

All measurements were appended to a common CSV file, providing a consolidated dataset for subsequent processing and analysis.

Each recorded observation contained the following fields:

- **Experiment identification:** The `timestamp`, `experiment`, `towers`, `shared_ledgers`, and `container` fields identified the time of observation, the active topology configuration, and the measured container.
- **CPU utilization:** The `cpu_percent` field recorded the percentage of CPU utilization reported by Docker for the container.
- **Memory consumption:** The `memory_bytes` field recorded container memory usage in bytes.
- **Network traffic:** The `network_rx_bytes` and `network_tx_bytes` fields recorded the cumulative network data received and transmitted by the container, respectively.
- **Disk I/O:** The `block_read_bytes` and `block_write_bytes` fields recorded the block I/O measurements reported for the container.

Where necessary, memory, network, and disk I/O measurements were converted from Docker's human-readable units into bytes to maintain a consistent numerical representation within the dataset.

The monitoring process collected measurements throughout container execution, including deployment-related activity, operational activity, and the subsequent settling period. Containers that had not yet started or were no longer running did not contribute samples during those intervals.

Because observations were labeled with their associated experiment identifier and topology parameters, the collected dataset enabled resource utilization to be examined both at the level of individual containers and across different network configurations.

This provided the experimental basis for investigating the relationship between network scale, shared-ledger configuration, and the computational overhead introduced by the proposed prototype.

4.2 Results

We now briefly go over the results of the previously described experiment above. The dataset created subsequently by the experiment contains more than 750000 records with each record holding six different measurements of interest. Due to the enormous nature of the dataset and its high dimensionality, we will only quote a subset of the records and represent them in three tables, table 2, table 3, and table 4. The reader can find the full dataset in it's entirety in the public repository [2].

Time	Container	Exp.	Towers	Ledgers	CPU (%)	Memory (Bytes)
11:43:18	vnf-1	1	2	1	0.00	81400955
11:43:18	nr_ue_2	1	2	1	0.04	3289383
11:43:18	nr_gnb_2	1	2	1	0.03	4690280
11:43:18	nr_ue_1	1	2	1	0.05	3948937
11:43:18	nr_gnb_1	1	2	1	0.03	4927259
11:43:18	upf	1	2	1	0.00	29852959
11:43:18	smf	1	2	1	0.00	42372956
11:43:18	amf	1	2	1	0.04	46839890
11:43:18	pcf	1	2	1	0.00	19870515
11:43:18	udr	1	2	1	0.00	7589593
11:43:18	udm	1	2	1	0.00	7118782
11:43:18	bsf	1	2	1	0.00	6476005

[2] `https://github.com/SamvitPrakash/Blockchain-Logging-Prototype`

Time	Container	Exp.	Towers	Ledgers	CPU (%)	Memory (Bytes)
11:43:18	nssf	1	2	1	0.00	6274679
11:43:18	ausf	1	2	1	0.00	10211033
11:43:18	scp	1	2	1	0.00	8012169
11:43:18	webui	1	2	1	0.00	220305818
11:43:18	nrf	1	2	1	0.00	19493028
11:43:18	mongo	1	2	1	0.21	130442854
11:43:18	grafana	1	2	1	0.07	115553075
11:43:18	metrics	1	2	1	0.08	27996979
11:43:22	vnf-1	1	2	1	0.02	87178609
11:43:22	nr_ue_2	1	2	1	0.04	3289383
11:43:22	nr_gnb_2	1	2	1	0.03	4690280
11:43:22	nr_ue_1	1	2	1	0.03	3948937
11:43:22	nr_gnb_1	1	2	1	0.03	4927259
11:43:22	upf	1	2	1	0.00	29580329
11:43:22	smf	1	2	1	0.05	42886758
11:43:22	amf	1	2	1	0.02	46326088
11:43:22	pcf	1	2	1	0.02	19870515
11:43:22	udr	1	2	1	0.02	7851737
11:43:22	udm	1	2	1	0.02	7380926
11:43:22	bsf	1	2	1	0.02	6738149
11:43:22	nssf	1	2	1	0.02	6536823
11:43:22	ausf	1	2	1	0.02	10473177
11:43:22	scp	1	2	1	0.17	9281995
11:43:22	webui	1	2	1	0.00	220305818
11:43:22	nrf	1	2	1	0.08	19786629
11:43:22	mongo	1	2	1	0.21	129603994
11:43:22	grafana	1	2	1	0.02	115028787
11:43:22	metrics	1	2	1	0.00	28017951

Table 2: CPU and memory utilization for Experiment 1 (9 September 2026, UTC+02:00).

Time	Container	RX (Bytes)	TX (Bytes)
11:43:18	vnf-1	10900	5370
11:43:18	nr_ue_2	16200	7820
11:43:18	nr_gnb_2	21500	12500
11:43:18	nr_ue_1	16400	7840

Time	Container	RX (Bytes)	TX (Bytes)
11:43:18	nr_gnb_1	21900	12600
11:43:18	upf	25100	58600
11:43:18	smf	47900	122000
11:43:18	amf	57700	124000
11:43:18	pcf	48000	53700
11:43:18	udr	47200	20300
11:43:18	udm	35300	25100
11:43:18	bsf	25200	8550
11:43:18	nssf	19200	6770
11:43:18	ausf	26400	15100
11:43:18	scp	245000	222000
11:43:18	webui	340000	23100
11:43:18	nrf	76900	111000
11:43:18	mongo	34300	38300
11:43:18	grafana	9290000	99600
11:43:18	metrics	285000	52000
11:43:22	vnf-1	11000	5500
11:43:22	nr_ue_2	16900	8090
11:43:22	nr_gnb_2	22300	12800
11:43:22	nr_ue_1	17100	8110
11:43:22	nr_gnb_1	22700	12800
11:43:22	upf	25600	58700
11:43:22	smf	49000	128000
11:43:22	amf	58400	125000
11:43:22	pcf	49300	56600
11:43:22	udr	47800	20700
11:43:22	udm	35900	25500
11:43:22	bsf	25800	8950
11:43:22	nssf	19800	7170
11:43:22	ausf	27000	15500
11:43:22	scp	249000	226000
11:43:22	webui	341000	23400
11:43:22	nrf	80500	112000
11:43:22	mongo	34700	38300
11:43:22	grafana	9290000	99600
11:43:22	metrics	293000	53800

Time	Container	RX (Bytes)	TX (Bytes)

Table 3: Network receive and transmit measurements for Experiment 1 (9 September 2026, UTC+02:00; 2 towers and 1 shared ledger).

Time	Container	Block Read (Bytes)	Block Write (Bytes)
11:43:18	vnf-1	50600000	12300
11:43:18	nr_ue_2	0	0
11:43:18	nr_gnb_2	0	0
11:43:18	nr_ue_1	397000	0
11:43:18	nr_gnb_1	1380000	0
11:43:18	upf	1460000	127000
11:43:18	smf	12100000	65500
11:43:18	amf	3620000	0
11:43:18	pcf	8140000	0
11:43:18	udr	238000	0
11:43:18	udm	520000	0
11:43:18	bsf	315000	0
11:43:18	nssf	319000	0
11:43:18	ausf	2300000	0
11:43:18	scp	2770000	0
11:43:18	webui	44100000	1510000
11:43:18	nrf	7660000	0
11:43:18	mongo	32300000	6640000
11:43:18	grafana	24400000	152000000
11:43:18	metrics	0	61400
11:43:22	vnf-1	52800000	12300
11:43:22	nr_ue_2	0	0
11:43:22	nr_gnb_2	0	0
11:43:22	nr_ue_1	397000	0
11:43:22	nr_gnb_1	1380000	0
11:43:22	upf	1460000	127000
11:43:22	smf	12100000	65500
11:43:22	amf	3620000	0
11:43:22	pcf	8140000	0
11:43:22	udr	238000	0

Time	Container	Block Read (Bytes)	Block Write (Bytes)
11:43:22	udm	520000	0
11:43:22	bsf	315000	0
11:43:22	nssf	319000	0
11:43:22	ausf	2300000	0
11:43:22	scp	2770000	0
11:43:22	webui	44100000	1510000
11:43:22	nrf	7660000	0
11:43:22	mongo	32300000	6640000
11:43:22	grafana	24400000	152000000
11:43:22	metrics	0	61400

Table 4: Block read and write measurements for Experiment 1 (9 September 2026, UTC+02:00; 2 towers and 1 shared ledger).

5 Critical Evaluation

We now proceed to perform a critical evaluation on the results gained from performing our experiments. We perform our critical evaluation by first constructing regression models, followed by the visualized analysis of the prototype's resource utilization, and finally a decision on the feasibility of blockchain logging within 5G Environments.

5.1 Computational Overhead Modeling

To evaluate the scalability of the proposed Blockchain Logging (BCL) prototype, regression models were constructed using the computational resource measurements obtained during experimentation. Separate models were fitted for CPU utilization, memory consumption, network reception, network transmission, block reads and block writes.

Each model expresses the predicted resource consumption as a function of the number of physical sites T and the number of shared ledgers L:

$$\hat{R}(T, L) = \beta_0 + \beta_1 T + \beta_2 L + \beta_3 TL$$

The coefficients were estimated using ordinary least squares regression

over aggregated experimental observations. The interaction term TL was included to capture potential relationships between site scaling and channel scaling.

Models were fitted separately for the individual prototype component roles, allowing the computational demands of decentralized BCL components to be distinguished from those of centralized BCLm infrastructure.

The resulting functions can be evaluated for both measured and unmeasured topology configurations.

Predictions within the experimentally observed parameter range provide an estimate of resource requirements under similar conditions. Predictions beyond this range represent mathematical extrapolations and are subject to additional uncertainty because they have not been experimentally validated.

To estimate deployment-wide resource requirements, the predicted per-component resource values are multiplied by the expected numbers of corresponding instances and then combined. These estimates depend on the deployment architecture, container allocation and component roles remaining consistent with the experimental prototype.

Consequently, the proposed models provide a quantitative planning mechanism for estimating the computational resource requirements of blockchain-based logging deployments, rather than a guarantee of production performance or capacity.

5.1.1 Interpolation and Extrapolation

The computational resource measurements collected during the experiments were used to construct a set of regression functions for evaluating the resource requirements of the Blockchain Logging (BCL) prototype. These functions relate the number of physical sites (towers) and the number of Hyperledger Fabric shared ledgers (channels) to resource consumption by individual prototype components. The resulting models provide estimates for experimental configurations that were not directly observed, as well as hypothetical deployment configurations larger than those used during the experiments.

For an individual component role c and measured resource m, the fitted

regression function is defined as

$$\widehat{R}_{c,m}(T, L) = \beta_{0,c,m} + \beta_{T,c,m}T + \beta_{L,c,m}L + \beta_{TL,c,m}TL, \tag{3}$$

where T denotes the number of towers, L the number of shared ledgers, and $\widehat{R}_{c,m}(T, L)$ the predicted mean resource usage for a container belonging to role c. The coefficients $\beta_{0,c,m}$, $\beta_{T,c,m}$, $\beta_{L,c,m}$ and $\beta_{TL,c,m}$ represent, respectively, the intercept, tower effect, channel effect and interaction between these two topology variables. The interaction term permits the relationship between resource demand and one topology parameter to vary with the value of the other parameter. These coefficients describe statistical associations within the prototype experiments and should not, on their own, be interpreted as causal effects.

The experiment automation considers topologies satisfying the assignment constraint given in Equation 1, which requires at least two peers per shared ledger under the experiment's configuration rule. The collected configurations range from 2 to 31 towers, with as many as 15 shared ledgers. In this study, interpolation refers to predicting resource consumption for an unmeasured feasible combination situated within the supported experimental topology region. Extrapolation refers to evaluating the same fitted function at a feasible combination beyond that region, such as a deployment of 50 or 100 towers. Importantly, the fact that each individual parameter is within its observed minimum and maximum does not guarantee that their combination was observed or is supported by comparable nearby observations.

No additional or independently trained extrapolation function is introduced: both interpolation and extrapolation use the same fitted coefficients of Equation 3. Extrapolated predictions must be treated as conditional estimates rather than experimentally verified resource requirements, since the functional relationship may change as deployment scale increases.

5.1.2 Fitted Computational Resource Functions

Separate regression functions were fitted for six measured resources—CPU utilization, memory consumption, network reception, network transmission, block reads and block writes—for each of the six steady-state prototype component roles: VNF, Fabric-Peer, Logging chaincode, prototype VNFM, Fabric-CA and Fabric-Orderer. These roles correspond to the decentralized BCL instances at physical sites and the centrally deployed BCLm management components. Transient enrollment and bootstrap containers are not

included in these steady-state functions because their principal purpose is provisioning rather than sustained operation. The prototype-specific VNFM discussed here must not be confused with an ETSI NFV-MANO VNFM.

Tables 5, 6, 7, 8, 9, and 10 give the fitted coefficients for each component role and resource, which can be substituted into equation 3 to calculate the prototype's resource requirements for specific component roles and resources.

The dependent variables are mean Docker CPU utilisation (percentage points), mean memory consumption (MiB), and mean network and block I/O rates (MiB/s). CPU percentage points represent the sum-compatible Docker CPU metric rather than an automatically normalised proportion of total host CPU capacity. The rate measurements are estimated from differences between successive cumulative Docker counters and therefore describe observed container I/O activity, not the amount of uniquely generated log data.

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	cpu_percent	3.294150818410116	0.09714548237625688	0.02033176782033911	-0.004112529921065491
fabric-orderer	cpu_percent	0.06707618300043178	0.023447869234657435	0.0038782995984115343	0.0002862058305445406
fabric-peer	cpu_percent	0.6715595263580006	0.0004122856433859152	-0.03618341574431097	0.0010330726626825975
logging	cpu_percent	0.5744573907912283	0.12312101959405827	-0.2805651080731137	0.0014891473523790666
vnf	cpu_percent	0.4720304325772436	0.03568004386833228	-0.04591673283180402	0.0005948063183968677
vnfm	cpu_percent	0.07445568148710861	0.2759733924150198	-0.03685780584683776	0.0067281527568280874

Table 5: Regression Coefficients for CPU Utilization (Docker CPU percentage points)

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	memory_mib	15.692351786172166	0.10992187644969058	-0.3644088738263604	0.0134786461215117
fabric-orderer	memory_mib	24.871283419866128	1.0709295828752705	0.0257807168226563	-0.02601219471362759
fabric-peer	memory_mib	58.08553097299097	-0.07123845151849452	-3.2292901540609837	0.06933814558391874
logging	memory_mib	54.249810627086966	0.1254217396667689	-0.31742450819037255	-0.0006685194115735316
vnf	memory_mib	25.923049148595343	0.01395327711547842	-0.2878734138027875	0.008459670157728603
vnfm	memory_mib	5.849517290563858	0.10451682711028745	-3.4238004708888194	0.1634574718540256

Table 6: Regression Coefficients for Memory Utilization (MiB)

5.1.3 Total Deployment Resource Requirements

The fitted component functions estimate resource usage at the level of a component role. For practical deployment planning, these predictions must be combined according to the number of active instances of each role. Let

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	network_rx_mib_s	0.00016790157668555443	8.875083031811881e-05	-7.356626203347167e-05	1.297760486582871e-06
fabric-orderer	network_rx_mib_s	0.0013533036687281986	0.0006584662813293939	-0.0008742649453859147	4.201838686565334e-05
fabric-peer	network_rx_mib_s	0.0059183401282039754	1.0768389961508684e-05	-0.000859502077940123	1.8990552729785694e-05
logging	network_rx_mib_s	0.00015098916886847481	5.01643608635617e-05	3.545817947924613e-05	-3.7797569341221443e-06
vnf	network_rx_mib_s	0.0009014149396014428	-9.641055070797995e-06	-4.930189039051242e-05	2.2260595264444802e-06
vnfm	network_rx_mib_s	0.0006251415330135172	9.311363235344584e-05	0.00010336598575877122	2.1051469407819783e-05

Table 7: Regression Coefficients for Network Reception (MiB/s)

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	network_tx_mib_s	0.0002574592021051545	0.00014996240496150385	-0.00011076923367179134	1.8867587523629568e-06
fabric-orderer	network_tx_mib_s	0.017548772677582532	0.0036398247454238757	-0.01061464209235246	0.000103461946614041 5
fabric-peer	network_tx_mib_s	0.0022239925310726764	-2.6276129249956323e-05	-7.734526346603382e-05	3.877473637197717e-06
logging	network_tx_mib_s	0.00021021073817319 65	-1.8445296123253076e-06	6.538761445314048e-06	1.1018735733286447e-09
vnf	network_tx_mib_s	0.001221819152790055	-1.6833920241391037e-05	-4.874861550926781e-05	2.665785625857425e-06
vnfm	network_tx_mib_s	0.0006349597000723154	9.792489167518315e-05	0.00012216809634227414	2.2971456197476246e-05

Table 8: Regression Coefficients for Network Transmission (MiB/s)

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	block_read_mib_s	1.9834387380474686e-05	4.483117481613207e-06	-4.2606674758064986e-05	2.159450633200443e-06
fabric-orderer	block_read_mib_s	-0.0018975975845378023	0.0003249467970432173	-0.0011714420130351458	2.7852954418663436e-05
fabric-peer	block_read_mib_s	-7.336713310721163e-05	7.920999742591081e-06	-2.471919789512695e-06	-3.960585188286362e-07
logging	block_read_mib_s	-2.12306378348218e-05	3.1401196516053736e-06	-2.4812713537399324e-06	-1.5074035002689302e-07
vnf	block_read_mib_s	-4.674976887858579e-05	7.395233435503871e-06	-1.602281741343326e-05	1.790186690620555e-07
vnfm	block_read_mib_s	-0.00185107156752334	0.0003134058686172544 6	-0.0004985961257880114	4.035802374200499e-06

Table 9: Regression Coefficients for Block Reads (MiB/s)

Role	Metric	b_0	b_{towers}	b_{ledgers}	$b_{\text{interaction}}$
fabric-ca	block_write_mib_s	0.016827128160649768	0.008905555781140755	-0.006128942749626042	9.36235232141952e-05
fabric-orderer	block_write_mib_s	0.023445617337263625	-7.318408521094064e-05	0.019336362791417164	-4.289196010107978e-06
fabric-peer	block_write_mib_s	0.018008898963963238	-0.00016495094095816091	0.001136397396752755	-2.2454581814042134e-05
logging	block_write_mib_s	-1.3877264526997274e-05	1.2231543320022662e-06	6.50498443362049e-07	-9.834400151039646e-08
vnf	block_write_mib_s	8.869320875987329e-09	6.613159958098103e-10	-1.1059606187052302e-08	4.4381339636313385e-10
vnfm	block_write_mib_s	0.0	0.0	0.0	0.0

Table 10: Regression Coefficients for Block Write (MiB/s)

$n_c(T, L)$ represent the number of concurrently running containers belonging to role c in the proposed deployment. In the prototype, one container of each of the VNF, Peer and Log roles runs at every physical site, while the VNFM, CA and Orderer roles each run once at the central data center, so that $n_c(T, L) = T$ for $c \in \{\text{VNF}, \text{Peer}, \text{Log}\}$ and $n_c(T, L) = 1$ for $c \in \{\text{VNFM}, \text{CA}, \text{Ord}\}$. For a resource metric m, the aggregate predicted requirement is

$$\widehat{R}_{\text{total},m}(T, L) = \sum_{c \in \mathcal{R}} n_c(T, L) \widehat{R}_{c,m}(T, L), \tag{4}$$

where $\mathcal{R} = \{\text{VNF}, \text{Peer}, \text{Log}, \text{VNFM}, \text{CA}, \text{Ord}\}$. The sum is valid as an estimate of total resource use when the per-role predictions use compatible measurement units and representative sampling windows. It does not automatically imply that the full amount constitutes additional overhead over normal 5G operations.

As an illustration, consider a deployment of $T = 50$ towers sharing $L = 10$ ledgers, which lies beyond the 31 towers evaluated experimentally and is therefore an extrapolation. Evaluating Equation 3 with the coefficients in Tables 5 and 6 gives per-instance CPU utilizations of approximately 2.1 (VNF), 0.8 (Peer), 4.7 (Log), 16.9 (VNFM), 6.3 (CA) and 1.4 (Ord) percentage points, and per-instance memory consumptions of approximately 28.0, 56.9, 57.0, 58.6, 24.3 and 65.7 MiB, respectively. Applying Equation 4, the aggregate prediction is approximately 405 CPU percentage points and 7243 MiB (about 7.1 GiB) of memory. In both cases the site-level roles account for most of the total (about 94% of the CPU and 98% of the memory), even though the per-instance requirements of some of the centralized components grow faster with deployment size.

5.2 Visualization and Analysis of Resource Utilization

To complement the regression models presented in the preceding subsection, a series of diagrams was generated using Python to visualize the relationship between deployment size and resource utilization. Once again, the python code is available together with the other artifacts generated throughout our research in a public repository [3]. While the regression equations provide a mathematical representation of resource consumption, the diagrams offer a graphical interpretation of how the resource requirements of individual components change as the blockchain logging infrastructure scales.

[3] `https://github.com/SamvitPrakash/Blockchain-Logging-Prototype`

The analysis considers six resource metrics: CPU utilization, memory consumption, network reception, network transmission, block reads, and block writes. Each metric is examined across the six component roles of the prototype: the Hyperledger Fabric Certificate Authority (CA), orderer, peer, logging component, VNF, and VNFM. The legends of the diagrams additionally contain the transient fabric-enroll container, which is shown for completeness but is excluded from the regression models.

The number of towers and shared ledgers constitute the two independent variables of the regression models. As established previously, the predicted resource consumption of a component is represented by a linear model incorporating both variables and their interaction. Consequently, the diagrams should be interpreted as visual representations of the resource requirements associated with the deployment configurations, rather than as functions of the number of towers alone.

The experimental configurations are subject to the constraint that each shared ledger must be associated with at least two peers, as given in Equation 1. This constraint is important when interpreting the diagrams, as it limits the combinations of towers and shared ledgers represented by the experimental measurements. Furthermore, where measurements from multiple ledger configurations are displayed against the same tower count, differences in resource consumption may reflect variations in the number of shared ledgers as well as the number of towers.

The following sub subsections discuss the resource metrics individually, highlighting their significance for the scalability and operational requirements of the proposed blockchain logging infrastructure.

5.2.1 CPU Utilization

CPU utilization represents the processing resources required by each component of the blockchain logging infrastructure. It is an important metric when evaluating scalability, as increasing computational requirements may ultimately constrain the number of components that can be accommodated by the underlying physical infrastructure.

Figure 5 illustrates CPU utilization for the different component roles across the experimental deployment configurations.

The distinction between component roles is particularly relevant because

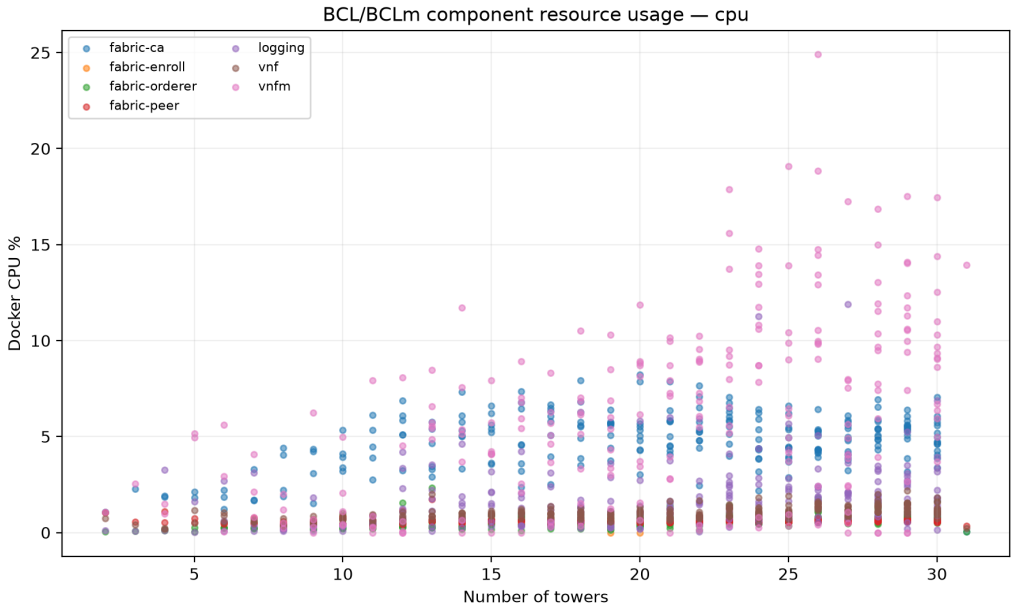


Figure 5: CPU utilization of the blockchain logging infrastructure components across the experimental deployment configurations.

each performs a different function within the prototype. The Fabric CA is responsible for identity management and certificate provisioning, while the orderer coordinates transaction ordering and block creation. Fabric peers maintain ledger state and participate in transaction processing, whereas the logging component provides the application-level functionality required to record network logs on the blockchain. The VNF and VNFM support the operation and management of the deployed infrastructure.

Most observations across all roles lie below approximately 10 percentage points, with the exception of the prototype VNFM, whose observations reach approximately 25 percentage points at the larger deployment sizes. The VNFM also has the largest tower coefficient in Table 5 (0.276 percentage points per additional tower), followed by the logging component (0.123) and the Fabric-CA (0.097), whereas the Fabric-Peer is almost insensitive to the number of towers (0.0004). The ledger coefficients are negative for all roles except the Fabric-CA and Fabric-Orderer, and the largest in magnitude belongs to the logging component (-0.281), so that adding ledgers at a fixed tower count reduces its estimated CPU utilization.

5.2.2 Memory Consumption

Memory consumption represents the amount of system memory required to maintain the execution state and supporting data structures of the individual components. Unlike CPU utilization, which describes processing activity, memory consumption provides an indication of the resources allocated to sustaining the operation of the infrastructure.

Figure 6 presents the memory consumption of the different component roles across the experimental deployment configurations. Memory values are expressed in mebibytes (MiB), allowing the relative requirements of individual components to be compared.

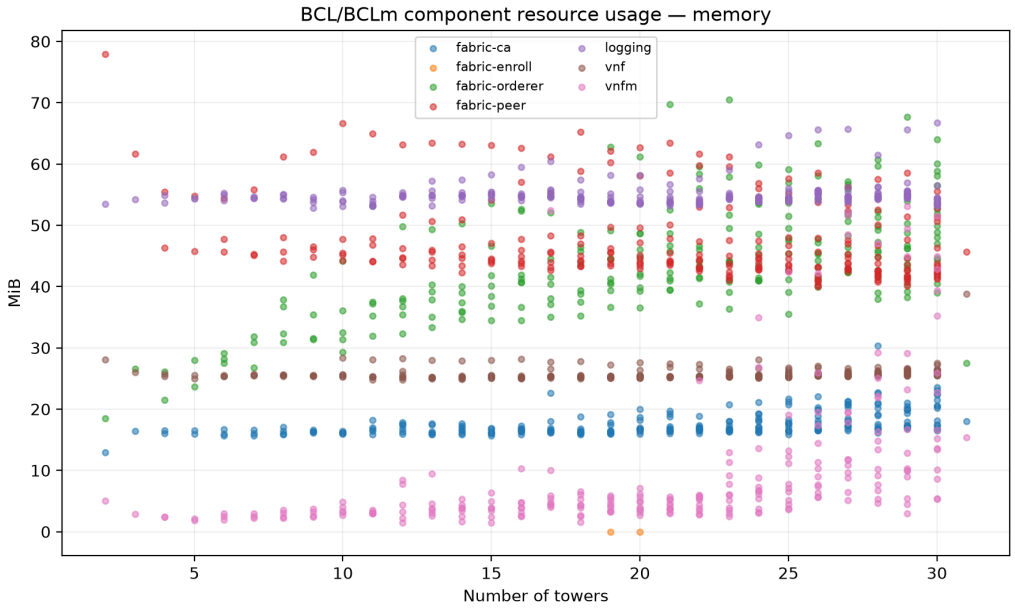


Figure 6: Memory consumption of the blockchain logging infrastructure components across the experimental deployment configurations.

Memory consumption is particularly relevant to the deployment of blockchain-based services within an NFV environment, where multiple virtualized components may compete for the memory resources of the same physical host.

The Fabric-Peer and logging components have the largest per-instance memory footprints, between roughly 40 and 60 MiB, and remain approximately constant as the number of towers grows (tower coefficients of -0.071 and 0.125 MiB per tower in Table 6). The Fabric-CA and VNF remain at roughly

15 to 30 MiB. The Fabric-Orderer is the only role with substantial growth, with a tower coefficient of 1.07 MiB per tower, while the VNFM grows more modestly from a small base. For the Fabric-Peer and the VNFM, the ledger coefficients are large and negative (−3.23 and −3.42 MiB per ledger) and are partially offset by positive interaction terms.

5.2.3 Network Reception

Network reception measures the volume of data received by the individual components over the network. It provides an indication of the incoming communication requirements of the blockchain logging infrastructure and is therefore relevant when assessing the network overhead introduced by the proposed solution.

Figure 7 illustrates network reception for the different component roles across the experimental deployment configurations.

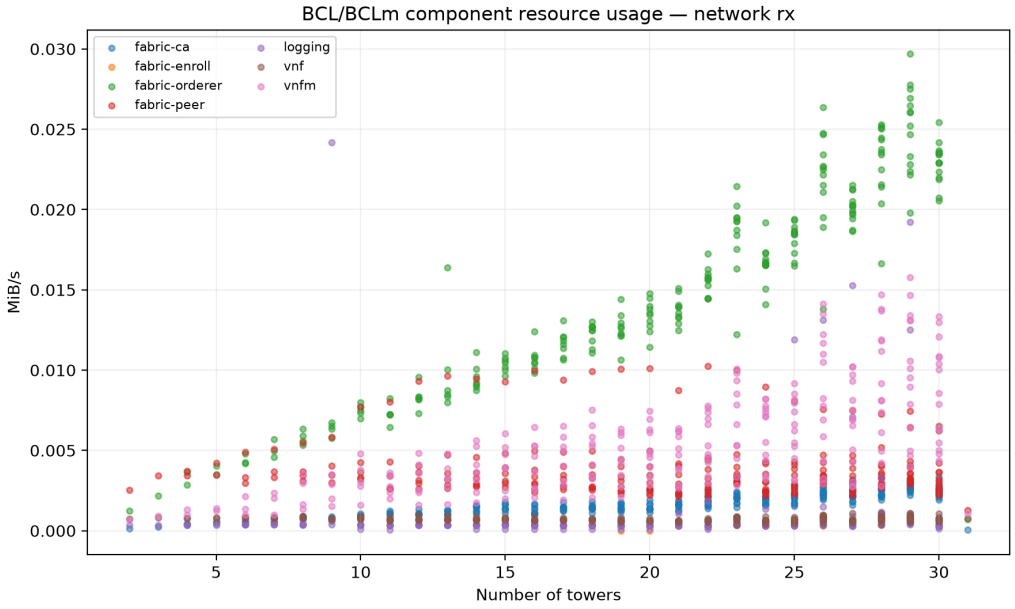


Figure 7: Network reception of the blockchain logging infrastructure components across the experimental deployment configurations.

Network reception is an important characteristic of a distributed blockchain network because communication is required between multiple components to support transaction submission, ordering, block dissemination, and ledger

maintenance.

Reception rates are low for all roles, remaining below approximately 0.03 MiB/s. The Fabric-Orderer is the only role that stands out, increasing with the number of towers (6.6×10^{-4} MiB/s per additional tower, the largest tower coefficient in Table 7), while the remaining roles stay below approximately 0.01 MiB/s. The negative ledger coefficient of the orderer (-8.7×10^{-4}), together with its positive interaction term, indicates that the effect of additional towers on its reception depends on the number of ledgers in the deployment.

5.2.4 Network Transmission

Network transmission describes the volume of data sent by the individual components of the blockchain logging infrastructure. Together with network reception, this metric provides a more complete representation of the communication resources required by the prototype.

Figure 8 presents network transmission for each component role across the experimental deployment configurations.

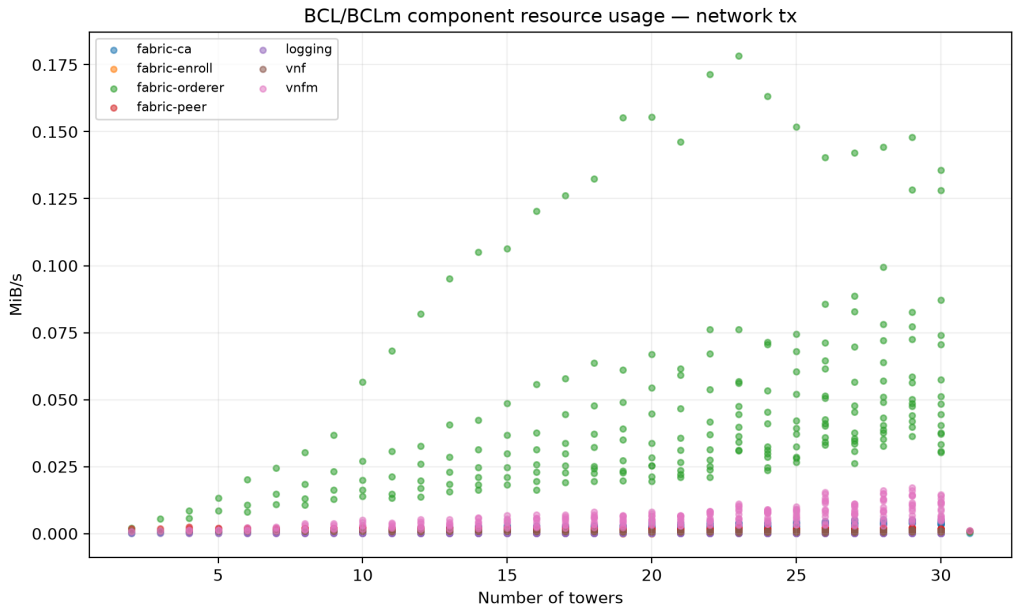


Figure 8: Network transmission of the blockchain logging infrastructure components across the experimental deployment configurations.

The transmission requirements of a component depend on its responsibilities within the distributed infrastructure. For example, components involved in blockchain transaction processing may transmit transaction proposals, responses, blocks, or other coordination messages, depending on their specific role in the Fabric network.

As with reception, transmission is dominated by the Fabric-Orderer, which reaches roughly 0.1 MiB/s at the upper end of the tested range, with isolated higher observations, whereas all other roles remain below approximately 0.02 MiB/s. The tower coefficient of the orderer in Table 8 is 0.0036 MiB/s per tower, while its ledger coefficient is negative (-0.0106), so that at a given tower count its transmission is highest for deployments with few ledgers.

5.2.5 Block Read Operations

Block read activity represents the amount of data read from the storage subsystem by each component. This metric provides an indication of the read-related storage requirements associated with the operation of the blockchain logging infrastructure.

Figure 9 illustrates block read activity for the different component roles across the experimental deployment configurations.

Storage activity is particularly relevant to the proposed solution because blockchain-based logging requires persistent storage and subsequent retrieval of ledger information.

Block read activity is negligible for almost all observations, remaining close to zero (below approximately 0.01 MiB/s) across the tested range. The isolated spikes visible in Figure 9, reaching approximately 0.09 and 0.17 MiB/s, belong to the orderer at the largest tower counts, while the remaining outliers are VNFM observations below approximately 0.05 MiB/s. Consistent with this, all coefficients in Table 9 are of the order of 10^{-3} or smaller, and the largest tower coefficients belong to the orderer and the VNFM.

5.2.6 Block Write Operations

Block write activity measures the amount of data written to the storage subsystem by each component. This metric is particularly relevant to blockchain-based logging, as the persistence of log records ultimately requires information to be written to storage.

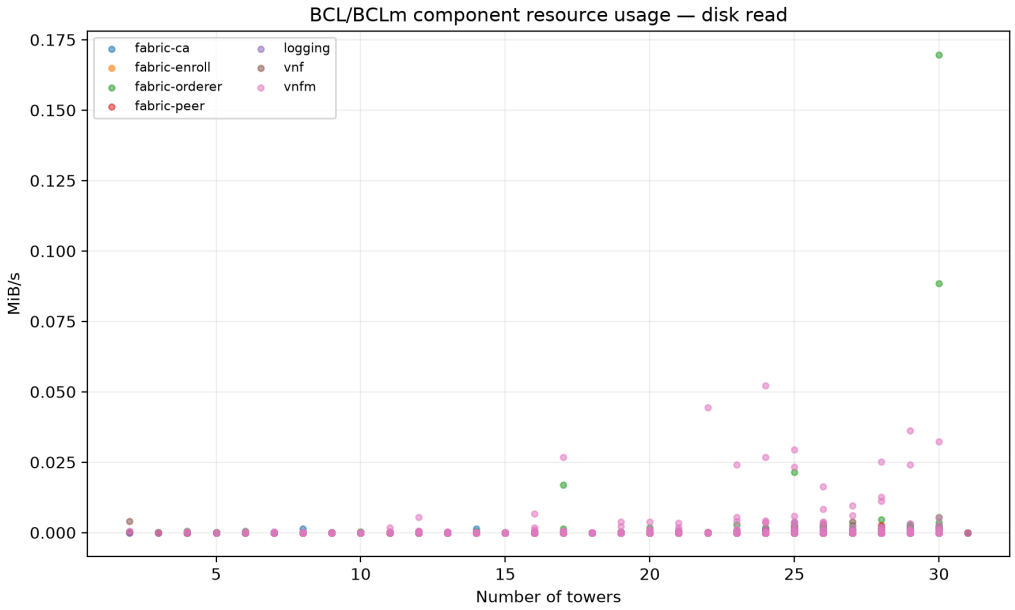


Figure 9: Block read activity of the blockchain logging infrastructure components across the experimental deployment configurations.

Figure 10 illustrates block write activity for the different component roles across the experimental deployment configurations.

The storage requirements of the prototype are influenced by the responsibilities of the individual components. Fabric peers maintain ledger information, while other components may perform write operations associated with configuration data, application state, logs, or other operational information.

Block write activity is dominated by the Fabric-CA and Fabric-Orderer, whose observations rise to approximately 0.3 MiB/s at the upper end of the tested range. For the CA the growth is driven by the number of towers (0.0089 MiB/s per tower in Table 10), whereas for the orderer it is driven mainly by the number of ledgers (0.0193 MiB/s per ledger), with a tower coefficient close to zero. The Fabric-Peer remains at approximately 0.02 MiB/s regardless of deployment size, and the logging component and VNF write virtually nothing.

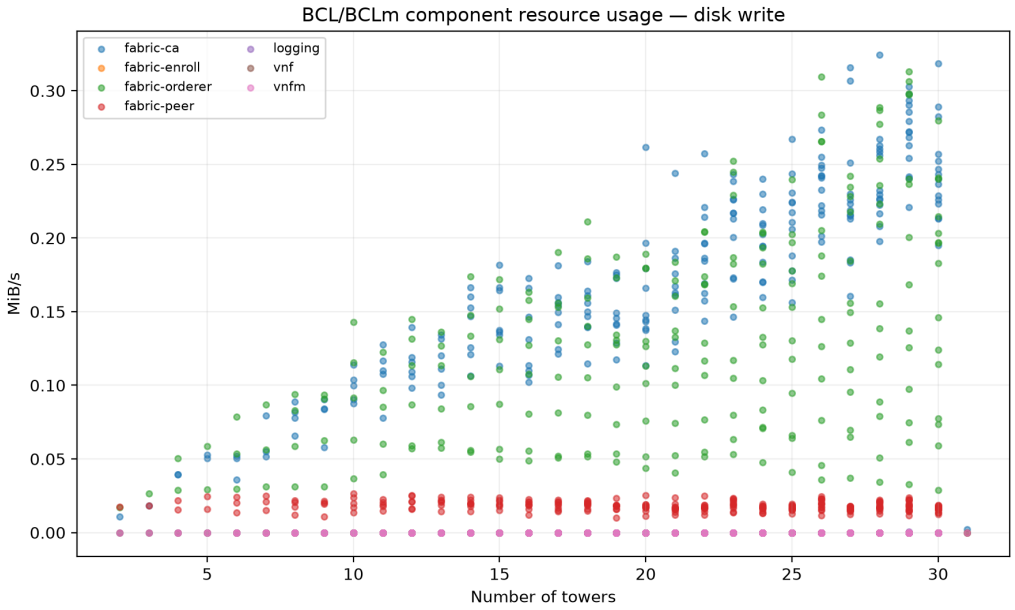


Figure 10: Block write activity of the blockchain logging infrastructure components across the experimental deployment configurations.

5.2.7 Overall Resource Utilization and Scalability

The six resource utilization diagrams collectively provide a graphical representation of the computational, memory, communication, and storage requirements of the blockchain logging infrastructure.

Across the six metrics, the roles whose per-instance requirements grow with deployment size are largely the centralized BCLm components, namely the Fabric-Orderer (memory, network and block writes), the Fabric-CA (CPU and block writes) and the VNFM (CPU), together with the CPU utilization of the logging component. The remaining site-level components, in particular the Fabric-Peer, the VNF and the memory of the logging component, remain approximately constant per instance, so that their contribution to the total grows through the number of instances rather than through their individual footprint.

When interpreting the diagrams, it is also important to distinguish between the resource consumption of an individual component instance and the aggregate requirements of the complete deployment. Even where the resource consumption of an individual instance changes only slightly, the total resource

requirements may increase as additional component instances are introduced.

The aggregation approach described in the preceding subsection addresses this distinction by combining the predicted resource consumption of individual component instances according to the composition of the deployment.

Finally, the diagrams must be interpreted within the limitations of the experimental environment and regression methodology. The visualizations describe resource utilization across the measured configurations, while the regression equations provide estimates based on the relationships identified in those measurements.

Predictions outside the experimental range are extrapolations and may not capture nonlinear scaling, resource saturation, changes in workload, or other effects that could emerge in larger deployments.

Nevertheless, the combined use of experimental visualizations and regression modeling provides a systematic basis for understanding the resource requirements of the prototype. This approach supports the estimation of infrastructure requirements for different deployment configurations and provides insight into the practical scalability of the proposed blockchain-based logging solution.

5.3 Overall Feasibility Assessment

Based on the experimental results and subsequent resource overhead modeling, this study concludes that the proposed blockchain-based logging prototype is computationally feasible for integration into a telecommunications network. However, feasibility cannot be adequately represented by a single resource utilization threshold or a universally optimal blockchain configuration, as different organizations operate under different business objectives, infrastructure limitations, performance requirements, and levels of risk tolerance. This perspective is supported by Afraz et al. [3], who demonstrate that blockchain feasibility is inherently dependent on the requirements of the intended application and the infrastructure supporting its deployment, and who emphasize the difficulty of establishing a universal optimization framework due to the heterogeneity of telecommunications applications and the trade-offs between resource utilization, performance, scalability, and cost. The experimental results reinforce this argument, as the computational overhead introduced by the prototype is not a fixed characteristic, but varies ac-

cording to the deployed network topology. The predictive models developed from these measurements therefore allow the resource implications of a proposed deployment to be assessed before its implementation, provided that the configuration falls within the applicable limits of the models, and enable organizations to select a topology according to their own computational resources and operational requirements.

Nevertheless, the conclusion of feasibility must be interpreted within the scope of the evaluation. The experiments characterize the resource overhead associated with the deployment life cycle and successful commitment of a known transaction under the tested conditions, with each topology executed once. Accordingly, the resulting models do not independently establish feasibility under sustained transaction workloads, nor do they account for all factors affecting production deployment, such as transaction throughput, latency, long-term ledger growth, infrastructure costs, and network reliability. Furthermore, predictions outside the experimentally evaluated parameter ranges require additional validation. Ultimately, this study concludes that the proposed blockchain logging prototype is feasible from a computational resource perspective, with its suitability for a particular deployment dependent on the organization's requirements and the selected network topology.

6 Conclusion and Future Work

6.1 Conclusion

The increasing complexity of fifth-generation (5G) telecommunications networks, particularly through the adoption of Network Function Virtualization (NFV), network slicing, and edge computing, introduces significant challenges for digital forensics and digital forensic readiness. The distributed and dynamic nature of these environments creates a need for mechanisms capable of preserving the integrity and validity of logs that may subsequently be required as digital evidence. This research investigated the feasibility of blockchain-based logging as a means of addressing these challenges within 5G NFV environments.

To investigate this feasibility, a blockchain-based logging prototype was designed and implemented using Hyperledger Fabric. The proposed architecture consists of decentralized Blockchain Logging (BCL) components deployed at individual physical sites and a centralized Blockchain Logging management (BCLm) component responsible for coordinating the blockchain

infrastructure. The prototype was integrated into a simulated 5G NFV environment, demonstrating the ability to deploy the proposed architecture and successfully commit transactions to shared blockchain ledgers.

The computational feasibility of the prototype was subsequently evaluated through systematic experimentation across multiple network topologies. By varying the number of physical towers and shared ledgers, measurements of CPU utilization, memory consumption, network reception, network transmission, block reads, and block writes were collected. These measurements were used to construct regression models capable of estimating the computational resource requirements of the individual prototype components as functions of the number of towers, shared ledgers, and their interaction.

The results of this investigation support the conclusion that blockchain-based logging is computationally feasible within the conditions evaluated by this research. However, the resource requirements of the proposed solution are dependent on the network topology in which it is deployed. Consequently, computational feasibility cannot be determined through a single universal resource threshold, as different telecommunications operators may have substantially different infrastructure capabilities, operational requirements, and security priorities.

An important contribution of this research is therefore the development of mathematical models that allow the computational overhead of the proposed architecture to be estimated for different deployment configurations. Rather than identifying a universally optimal network topology, these models provide a quantitative foundation through which organizations can evaluate the resource implications of alternative configurations and determine their suitability according to their individual constraints.

Nevertheless, the conclusion of computational feasibility remains subject to the limitations of the experimental environment. The experiments primarily evaluated the deployment life cycle and successful commitment of a known transaction, rather than sustained blockchain operation under realistic telecommunications logging workloads. Furthermore, each topology was executed once, and the regression models were derived from a finite range of experimental configurations, and predictions beyond these configurations require further validation.

Ultimately, this research demonstrates the practical potential of blockchain-based logging as a mechanism for supporting digital forensic readiness in 5G

NFV environments. Through the development of a functioning prototype, systematic experimentation, and predictive resource modeling, the study establishes a foundation for evaluating the computational requirements of such a solution. The research therefore contributes both a proposed architecture for preserving the integrity of NFV logs and a quantitative basis for investigating its suitability within different telecommunications environments.

6.2 Future Work

Although this research establishes the computational feasibility of the proposed blockchain-based logging prototype within the evaluated conditions, several opportunities remain for further investigation. In particular, the flexibility of the proposed architecture allows for numerous alternative blockchain network topologies, each potentially presenting different computational, operational and security trade-offs. Further research is therefore necessary to understand these trade-offs and establish how the architecture may be optimized for different telecommunications environments.

6.2.1 Investigation of Alternative Network Topologies

One of the most significant opportunities for future research concerns the investigation of alternative blockchain network topologies. The experiments conducted in this study considered configurations in which each Fabric peer participated in only one shared ledger at a time. Furthermore, peers were distributed between the available ledgers using a pseudo-random assignment mechanism, with each ledger requiring a minimum of two participating peers.

Although this approach provided a consistent and reproducible foundation for evaluating computational overhead, it represents only a subset of the possible network configurations supported by the proposed architecture.

An alternative topology could permit individual peers to participate in multiple shared ledgers simultaneously. For example, a physical tower identified as operationally critical could participate in two or more ledgers, providing additional ledger replication or enabling its logs to be recorded across multiple independently maintained channels. Such a configuration could potentially improve the availability or resilience of important forensic records, depending on how the ledgers, endorsement policies, and participating organizations are configured. However, participation in multiple ledgers would also introduce additional computational, memory, communication, and stor-

age requirements, which would need to be evaluated experimentally.

Another important research direction concerns the distribution of peers according to the security requirements of the infrastructure being protected. Rather than distributing peers approximately equally between shared ledgers, their allocation could be determined through a risk-based approach. Under such an approach, ledgers associated with high-priority or high-risk infrastructure could be assigned a greater number of participating peers, while ledgers associated with lower-risk infrastructure could operate with fewer peers.

For example, critical telecommunications infrastructure or network functions supporting sensitive services could benefit from greater ledger replication and more restrictive endorsement requirements. Conversely, infrastructure with lower security requirements could employ smaller peer groups to reduce resource consumption. This approach could enable the computational resources allocated to blockchain-based logging to reflect the security priorities of the telecommunications operator.

Future investigations could additionally consider heterogeneous topologies in which different ledgers have different numbers of peers, overlapping channel memberships, or distinct organizational participation requirements. Dynamic topologies, in which peer and channel assignments change in response to network conditions, resource availability, or changing security risk levels, may also offer opportunities for further optimization.

These alternative configurations introduce trade-offs between resource consumption, fault tolerance, availability, confidentiality, and forensic integrity. Importantly, increasing the number of participating peers does not automatically guarantee stronger security, as the security properties of a Hyperledger Fabric deployment also depend on its endorsement policies, ordering service, identity management, and the independence of participating infrastructure.

Consequently, further experimentation should investigate these characteristics alongside computational overhead to determine the practical benefits and limitations of each topology. The regression models developed in this research could serve as a foundation for these investigations, although additional topology parameters and new experimental measurements would be required to characterize configurations beyond those originally evaluated.

Ultimately, the large number of possible blockchain network topologies presents

an extensive area for future research. A more comprehensive understanding of their respective benefits, limitations, and resource requirements could enable telecommunications operators to select or dynamically construct configurations that balance forensic integrity, security requirements, and available computational resources.

6.2.2 Evaluation Under Sustained Logging Workloads

A second area for future research involves evaluating the prototype under sustained and realistic logging workloads. The experiments conducted in this study established the computational resource requirements associated with deploying the prototype and successfully committing a known transaction to the corresponding ledger for each tower.

However, operational telecommunications networks continuously generate logs from numerous network components, often at varying rates depending on network activity and operational conditions. Consequently, additional experiments are required to evaluate how the prototype performs when exposed to continuous log generation and transaction submission.

Future experiments could investigate transaction throughput, transaction commitment latency, resource utilization under sustained workloads, and the influence of increasing log volumes on blockchain performance. Different transaction submission rates and log sizes could also be evaluated to identify possible bottlenecks and resource saturation thresholds.

These investigations would complement the computational resource models developed in this study and provide a more comprehensive understanding of the prototype's operational feasibility in production telecommunications environments.

6.2.3 Long-Term Ledger Management and Storage Optimization

Another important area for future research concerns the long-term management of blockchain ledgers. The current prototype does not implement mechanisms for log archival and ledger rotation, despite the continuously growing storage requirements associated with append-only blockchain records.

In a production telecommunications network, continuous log generation would result in increasing storage demands over time. Consequently, future research should investigate mechanisms for separating long-term log storage from the

blockchain infrastructure while preserving the ability to verify the integrity and authenticity of archived records.

One possible approach would involve storing logs within a dedicated archival system while retaining cryptographic hashes or other verification information on the blockchain. Such an approach could reduce the amount of data maintained directly by blockchain peers while preserving the ability to detect unauthorized modifications to archived records.

Future experimentation should evaluate the computational and storage implications of these mechanisms, as well as their effects on forensic evidence retrieval and verification.

6.2.4 Integration into Production NFV Environments

Finally, future research should investigate the integration of the proposed architecture into production-oriented NFV environments. The simulated environment developed for this study abstracted the NFV Infrastructure (NFVI) and NFV Management and Orchestration (NFV-MANO) through Docker, Docker Compose, and custom automation scripts.

Although this approach was suitable for evaluating the prototype, a production deployment would require integration with established NFV management and orchestration mechanisms. This would include automated lifecycle management, secure credential provisioning, resource allocation, and reliable collection of logs from the relevant network functions.

Further investigations could also evaluate the prototype on geographically distributed infrastructure, where network latency, bandwidth limitations, component failures, and differences in available computational resources may influence blockchain performance.

Such research would provide a more realistic assessment of the operational challenges associated with deploying blockchain-based logging within telecommunications infrastructure and help determine the adaptations required for practical adoption.

Collectively, these future research directions would extend the findings of this study beyond computational feasibility towards a more comprehensive evaluation of operational performance, security, and deployment suitability. In particular, the investigation of alternative network topologies offers an

opportunity to develop blockchain logging architectures that can be tailored to the heterogeneous security requirements and resource constraints of 5G telecommunications networks.

References

[1] H. He, S. Fei, and Z. Yan, "Advancing 5g security and privacy with ai: A survey," *ACM Comput. Surv.*, vol. 58, Sept. 2025.

[2] M. Ghoshal, O. Basit, S. Wang, P. Dinh, I. Khan, Y. Feng, Z. Yu, Y. C. Hu, and D. Koutsonikolas, "A first large-scale study of operational 5g standalone networks," *Proc. ACM Netw.*, vol. 3, Nov. 2025.

[3] N. Afraz, F. Wilhelmi, H. Ahmadi, and M. Ruffini, "Blockchain and smart contracts for telecommunications: Requirements vs. cost analysis," *IEEE Access*, vol. 11, pp. 95653–95666, 2023.

[4] Z. Nezami, Z. Li, C. Qin, F. Banaie, R. Khalid, and E. Pournaras, "Blockchain and edge computing nexus: A large-scale systematic literature review," *Distrib. Ledger Technol.*, Mar. 2026. Just Accepted.

[5] A. Alghuried, M. Alkinoon, M. Mohaisen, A. Wang, C. Zou, and D. Mohaisen, "Blockchain security and privacy: Threats, challenges, applications, and tools," *Distrib. Ledger Technol.*, vol. 5, Dec. 2025.

[6] J. Dias, P. Pinto, R. Santos, and S. Malta, "5G network slicing: Security challenges, attack vectors, and mitigation approaches," *Sensors (Basel)*, vol. 25, June 2025.

[7] C. De Alwis, P. Porambage, K. Dev, T. R. Gadekallu, and M. Liyanage, "A survey on network slicing security: Attacks, challenges, solutions and research directions," *IEEE Communications Surveys Tutorials*, vol. 26, no. 1, pp. 534–570, 2024.

[8] A. K. Alnaim, "Securing 5g virtual networks: a critical analysis of sdn, nfv, and network slicing security," *International Journal of Information Security*, vol. 23, pp. 3569–3589, Dec 2024.

[9] M. Zoure, T. Ahmed, and L. Reveillere, "Network services anomalies in nfv: Survey, taxonomy, and verification methods," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, p. 1567–1584, 2022.

[10] European Union Agency for Cybersecurity (ENISA), "Nfv security in 5g: Challenges and best practices," tech. rep., European Union Agency for Cybersecurity (ENISA), Feb. 2022.

[11] F. Casino, T. K. Dasaklis, G. P. Spathoulas, M. Anagnostopoulos, A. Ghosal, I. Borocz, A. Solanas, M. Conti, and C. Patsakis, "Research trends, challenges, and emerging topics in digital forensics: A review of reviews," 2022.

[12] V. Machaka and T. Balan, "Investigating proactive digital forensics leveraging adversary emulation," *Applied Sciences*, vol. 12, no. 18, 2022.

[13] *Information technology — Security techniques — Incident investigation principles and processes*, 1 ed.

[14] G. Grispos, J. Garcia-Galan, L. Pasquale, and B. Nuseibeh, "Are you ready? towards the engineering of forensic-ready systems," 2017.

[15] N. M. Karie and C. Valli, "Digital forensic readiness implementation in sdn: Issues and challenges," 2021.

[16] F. Sharevski, "Towards 5G cellular network forensics," *EURASIP Journal on Information Security*, vol. 2018, p. 8, July 2018.

[17] S. Makura and H. Venter, "Towards the development of a digital forensic readiness model for 5g nfv environments," in *2024 IST-Africa Conference (IST-Africa)*, pp. 1–12, 2024.

[18] R. Zhao, M. Shoaib, V. T. Hoang, and W. U. Hassan, "Rethinking tamper-evident logging: A high-performance, co-designed auditing system," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, p. 2624–2638, ACM, Nov. 2025.

[19] W. Pourmajidi, L. Zhang, J. Steinbacher, T. Erwin, and A. Miranskyy, "Immutable log storage as a service on private and public blockchains," *IEEE Transactions on Services Computing*, vol. 16, p. 356–369, Jan. 2023.

[20] M. S. Islam and M. S. Rahman, "Logstamping: A blockchain-based log auditing approach for large-scale systems," 2025.

[21] A. J. Akbarfam, M. Heidaripour, H. Maleki, G. Dorai, and G. Agrawal, "Forensiblock: A provenance-driven blockchain framework for data forensics and auditability," 2023.

[22] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, p. 1–15, ACM, Apr. 2018.

[23] P. Thakkar, S. Nathan, and B. Vishwanathan, "Performance benchmarking and optimizing hyperledger fabric blockchain platform," 2018.

[24] C. Gorenflo, S. Lee, L. Golab, and S. Keshav, "Fastfabric: Scaling hyperledger fabric to 20,000 transactions per second," 2019.

[25] J. Sousa, A. Bessani, and M. Vukolić, "A byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform," 2017.

[26] European Telecommunications Standards Institute, *Network Functions Virtualisation (NFV); Architectural Framework*, Dec. 2014.

[27] "Open5gs: Open source 5gc and epc." `https://open5gs.org`. Accessed: 2026-10-13.

[28] S. Herle, "docker_open5gs." `https://github.com/herlesupreeth/docker_open5gs`. Accessed: 2026-10-13.

[29] 3rd Generation Partnership Project (3GPP), *NG-RAN; Xn general aspects and principles*.